Tree-Based Methods and Boosting

Lecture 3 ▪ POLSCI 689L

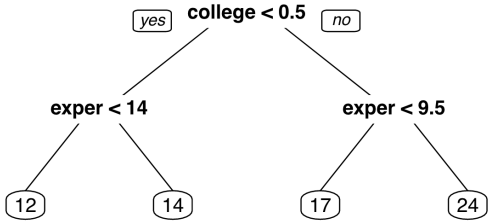
Jiawei Fu
Department of Political Science ▪ Duke University

Overview

1. **A single tree:** recursive binary splitting and regional predictions
2. **Tree complexity:** the bias–variance trade-off and cost-complexity pruning
3. **Bagging and random forests:** variance reduction, decorrelation, and out-of-bag evaluation
4. **Interpretation and inference:** feature importance and forest uncertainty
5. **Boosting:** sequential additive modeling, shrinkage, and early stopping

Why move beyond linear models?

- Linear models do not automatically capture nonlinear relationships between predictors and outcomes.
- We may also want to allow interactions among predictors.
- Higher-order and interaction terms can be added manually, but trees let the data reveal a hierarchical structure.



Key idea

A tree predicts by asking a sequence of simple questions.

A regression tree partitions the predictor space

Let $R_1, \ldots, R_J$ be distinct regions of the predictor space. The tree makes one prediction in each region:

$$\widehat{f}(x) = \sum_{j=1}^{J} \widehat{y}_{R_j} \mathbf{1}\{x \in R_j\}, \qquad \widehat{y}_{R_j} = \frac{1}{|R_j|} \sum_{i: x_i \in R_j} y_i.$$

The desired regions minimize the within-region residual sum of squares:

$$\sum_{j=1}^{J} \sum_{i: x_i \in R_j} (y_i - \widehat{y}_{R_j})^2.$$

- The leaves of the tree are the terminal regions R_j.
- The fitted function is piecewise constant over these regions.

Searching over all partitions is infeasible

- It is computationally infeasible to evaluate every possible partition of a p-dimensional predictor space into J regions.
- Trees therefore use *recursive binary splitting*: a top-down, greedy procedure.

Top-down

Start with all observations in one region and repeatedly divide the current regions into two.

Greedy

At each step, choose the locally best available split rather than reconsidering the full tree globally.

Key idea

The algorithm replaces a global combinatorial search with a sequence of local decisions.

Choosing one split

Select a predictor X_j and cut point s, producing

$$R_1(j,s) = \{x : x_j < s\}, \qquad R_2(j,s) = \{x : x_j \geq s\}.$$

The CART split minimizes

$$\min_{j,s} \left[\min_{c_1} \sum_{i:x_i \in R_1(j,s)} (y_i - c_1)^2 + \min_{c_2} \sum_{i:x_i \in R_2(j,s)} (y_i - c_2)^2 \right].$$

For a fixed (j,s), the optimal constants are the regional sample means:

$$\widehat{c}_k = \mathbb{E}_n[y_i \mid x_i \in R_k(j,s)], \qquad k = 1, 2.$$

Recursive binary splitting grows the tree

1. Search over predictors j and cut points s for the split that yields the largest reduction in residual sum of squares.
2. Replace the current region with the two daughter regions.
3. Repeat the splitting process within each resulting region.
4. Stop according to a tree-size rule, then assign each leaf its regional mean.

The central tuning problem

How large should the tree be?

Tree depth reproduces the bias–variance trade-off

Deep tree

- Many terminal regions
- Low training error
- Low bias and high variance
- Greater risk of overfitting

Shallow tree

- Few terminal regions
- Higher training error
- Higher bias and low variance
- May miss important structure

Key idea

Tree size controls flexibility in the same way that a regularization parameter does.

Why not stop as soon as a split looks weak?

A natural rule is to stop when the next split produces only a small reduction in residual sum of squares.

The problem

A seemingly unhelpful split near the top of the tree may enable a highly informative split later. A local stopping rule can therefore be too short-sighted.

A better strategy is:

1. grow a large tree, stopping only when a minimum leaf size is reached; and
2. prune the large tree back to an appropriate subtree.

Cost-complexity pruning penalizes tree size

For each tuning parameter $\alpha \geq 0$, choose a subtree $T \subseteq T_0$ that minimizes

$$\sum_{m=1}^{|T|} \sum_{i:x_i \in R_m} (y_i - \widehat{y}_{R_m})^2 + \alpha |T|,$$

where $|T|$ is the number of terminal nodes.

- The first term measures training error.
- The second term charges for each terminal node.
- Larger α favors a smaller subtree.
- Cross-validation selects α using prediction error on held-out folds.

The regression-tree workflow

Algorithm 8.1 *Building a Regression Tree*

1. Use recursive binary splitting to grow a large tree on the training data, stopping only when each terminal node has fewer than some minimum number of observations.

2. Apply cost complexity pruning to the large tree in order to obtain a sequence of best subtrees, as a function of α.

3. Use K-fold cross-validation to choose α. For each $k = 1, \ldots, K$:

   (a) Repeat Steps 1 and 2 on the $\frac{K-1}{K}$th fraction of the training data, excluding the kth fold.

   (b) Evaluate the mean squared prediction error on the data in the left-out kth fold, as a function of α.

   Average the results, and pick α to minimize the average error.

4. Return the subtree from Step 2 that corresponds to the chosen value of α.

Source: [James et al., 2013].

A single tree is interpretable but unstable

Advantages

- The decision rule is easy to interpret and resembles human decision-making.
- Splits are insensitive to monotone transformations of the predictors because they depend on their ordering.
- Trees perform automatic variable selection.

Disadvantages

- A small change in the data can produce a very different sequence of splits.
- Errors near the top propagate to all later branches.
- The fitted prediction surface is not smooth.

Key idea

Ensembles keep trees' flexibility while addressing their high variance.

Averaging reduces variance

If $Z_1, \ldots, Z_B$ are independent with variance σ^2, then

$$\mathrm{Var}\left(\frac{1}{B}\sum_{b=1}^{B} Z_b\right) = \frac{\sigma^2}{B}.$$

This suggests a general strategy:

1. obtain many training sets;
2. fit a high-variance model to each one; and
3. average the resulting predictions.

Key idea

We usually have only one training set, so the bootstrap supplies the repeated samples.

Bagging means bootstrap aggregation

Given one training sample:

1. Draw B bootstrap samples from the training data.
2. Fit a separate tree $\widehat{T}_b$ to each bootstrap sample.
3. Average the predictions:

$$\widehat{F}_{\text{bag}}(x) = \frac{1}{B}\sum_{b=1}^{B}\widehat{T}_b(x).$$

Bias and variance

Bagging is designed to reduce variance, not bias. The individual trees are therefore grown deep and are not pruned: each tree has low bias and high variance, while averaging stabilizes the ensemble.

Correlation limits the benefit of averaging

Suppose the trees have common variance σ^2 and positive pairwise correlation ρ. Then

$$\mathrm{Var}\left(\frac{1}{B}\sum_{b=1}^{B}\widehat{T}_b(x)\right) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2.$$

- As B grows, the second term vanishes but the first remains.
- If one predictor is very strong, many bagged trees may select it near the root and become highly correlated.
- Further variance reduction therefore requires *decorrelating* the trees.

A Bayesian alternative based on ensembles of trees is Bayesian additive regression trees (BART).

Random forests decorrelate the trees

At each split of each tree:

1. draw a fresh random subset of m predictors from the full set of p predictors;
2. search for the best split using only those m candidates; and
3. continue recursively until the leaf-size rule is reached.

A common choice in the source material is

$$m = \sqrt{p}.$$

Restricting the candidate predictors prevents the same strong variable from dominating the top split of every tree.

The random-forest estimator

A random forest averages B randomized tree estimators:

$$\widehat{F}(x) = \frac{1}{B} \sum_{b=1}^{B} \widehat{T}_b(x).$$

Each tree is based on:

- a subsample or bootstrap sample of size s;
- m randomly selected candidate predictors at each split; and
- a minimum leaf size k.

Key idea

The main tuning parameters are the sample size s, the candidate-set size m, and the leaf size k.

Out-of-bag observations provide internal validation

- A bootstrap sample does not contain every training observation.
- Observations not used to fit tree b are *out of bag* (OOB) for that tree.
- For observation i, average predictions only from trees for which i was OOB.
- Compare the resulting OOB prediction with y_i, then aggregate the errors over observations.

Why OOB error is useful

The OOB estimate is similar to an N-fold cross-validation estimate, but it is produced while the forest is being fitted.

Random forests can support asymptotic inference

Under regularity conditions, including honesty and subsampling with

$$s = n^{\beta}, \qquad \beta < 1,$$

[Wager and Athey, 2018] establish asymptotic normality:

$$\frac{\widehat{F}(x) - F(x)}{\sigma(x)} \longrightarrow N(0, 1).$$

- Honesty separates the observations used to select splits from those used to estimate leaf predictions.
- The result motivates standard errors and confidence intervals for forest predictions.

We return to honesty in later lectures.

Estimating uncertainty for a forest prediction

The source presents the following variance estimator:

$$\widehat{\sigma}(x) = \frac{n-1}{n}\left(\frac{n}{n-s}\right)\sum_{i=1}^{n}\left[\mathrm{Cov}\left(\widehat{T}_b(x), N_{ib}\right)\right]^2,$$

where N_{ib} indicates whether training observation i was used for tree b.

- The covariance measures how sensitive the forest prediction is to the inclusion of observation i.
- The contributions are aggregated over all training observations.

See [Wager and Athey, 2018].

LOCO importance removes one predictor

To assess the importance of feature X_j:

1. fit the forest using all predictors;
2. fit a restricted forest without access to X_j; and
3. compare their prediction errors.

If removing X_j barely changes predictive accuracy, its contribution is small.

Leave-Out-COvariates (LOCO)

Rank features according to the increase in prediction error when each feature is withheld.

Permutation importance: intuition

- Start with a trained model and a test set.
- Randomly permute feature X_j across test observations while leaving all other variables unchanged.
- Recompute prediction error using the permuted test data.
- A large increase in error indicates that the model relied strongly on X_j.

Key idea

Permutation preserves the feature's marginal values but breaks its association with the outcome and the other predictors.

Permutation importance: calculation

Let $\widehat{f}$ be the trained model and L the loss. First compute

$$\text{error}_{\text{orig}} = \frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} L(y_i, \widehat{f}(\boldsymbol{x}_i)).$$

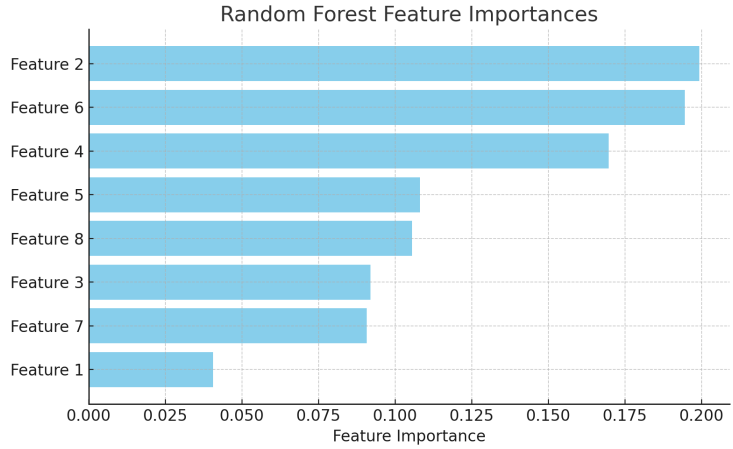
For each feature j:

1. permute X_j to obtain $\boldsymbol{x}_i^{\text{perm},j}$;
2. compute

$$\text{error}_{\text{perm}}^{j} = \frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} L(y_i, \widehat{f}(\boldsymbol{x}_i^{\text{perm},j}));$$

3. report either the difference or ratio relative to the original error.

A feature-importance ranking



Boosting learns an additive prediction rule

Consider the additive basis-function model

$$f(x) = \sum_{m=1}^{M} \beta_m b(x; \gamma_m).$$

The full optimization problem is

$$\min_{\{\beta_m, \gamma_m\}_{m=1}^{M}} \sum_{i=1}^{N} L\left(y_i, \sum_{m=1}^{M} \beta_m b(x_i; \gamma_m)\right),$$

which can be difficult to solve jointly.

- Boosting constructs the components sequentially.
- Each new weak learner focuses on what the current model has not yet learned well.

Forward stagewise additive modeling

Start with a simple initial function $f_0(x)$, such as 0, $\overline{y}$, or a generalized linear model. For $m = 1, \ldots, M$:

1. solve

$$(\beta_m, \gamma_m) = \arg\min_{\beta,\gamma} \sum_{i=1}^{N} L(y_i, f_{m-1}(x_i) + \beta b(x_i; \gamma))\,;$$

2. update

$$f_m(x) = f_{m-1}(x) + \beta_m b(x; \gamma_m).$$

The number of iterations M is a central tuning parameter.

With squared error, boosting fits the residuals

For squared-error loss,

$$L(y, f(x)) = (y - f(x))^2.$$

At step m,

$$\begin{aligned} & L(y_i, f_{m-1}(x_i) + \beta b(x_i; \gamma)) \\ & \quad = (y_i - f_{m-1}(x_i) - \beta b(x_i; \gamma))^2 \\ & \quad = (r_{im} - \beta b(x_i; \gamma))^2, \end{aligned}$$

where

$$r_{im} = y_i - f_{m-1}(x_i)$$

is the current residual. Thus the next weak learner is fitted to the residuals.

Shrinkage makes each boosting step smaller

Instead of taking a full update, use

$$f_m(x) = f_{m-1}(x) + \nu \beta_m b(x; \gamma_m), \qquad 0 < \nu \leq 1.$$

- The step size ν controls how much each learner changes the current prediction.
- A small value such as $\nu = 0.1$ is common in practice.
- If M is too large, boosting can overfit.
- Monitor validation performance and stop when it begins to deteriorate; this is *early stopping*.

Key idea

Boosting is controlled jointly by the number of iterations and the size of each update.

Boosted trees

A tree can be written as

$$T(x;\Theta) = \sum_{j=1}^{J} \widehat{y}_{R_j} \mathbf{1}\{x \in R_j\}, \qquad \Theta = \{R_j, \widehat{y}_{R_j}\}_{j=1}^{J}.$$

The tree parameters solve

$$\widehat{\Theta} = \arg\min_{\Theta} \sum_{i=1}^{N} L(y_i, T(x_i;\Theta)).$$

In a boosted tree model, each weak learner $b(x;\gamma_m)$ is replaced by a small tree $T(x;\Theta_m)$.

Boosted trees add one tree at a time

At iteration m, solve

$$\widehat{\Theta}_m = \arg\min_{\Theta} \sum_{i=1}^{N} L(y_i, f_{m-1}(x_i) + T(x_i; \Theta)),$$

then update

$$f_m(x) = f_{m-1}(x) + \nu T(x; \widehat{\Theta}_m).$$

- With squared-error loss, the new tree is fitted to the current residuals.
- Tree depth controls the complexity of each weak learner.
- The number of trees and the shrinkage parameter govern the overall fit.

Boosting for regression trees

Algorithm 8.2 *Boosting for Regression Trees*

1. Set $\hat{f}(x) = 0$ and $r_i = y_i$ for all i in the training set.
2. For $b = 1, 2, \ldots, B$, repeat:
   (a) Fit a tree $\hat{f}^b$ with d splits ($d+1$ terminal nodes) to the training data (X, r).
   (b) Update $\hat{f}$ by adding in a shrunken version of the new tree:
   $$\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}^b(x). \tag{8.10}$$
   (c) Update the residuals,
   $$r_i \leftarrow r_i - \lambda \hat{f}^b(x_i). \tag{8.11}$$
3. Output the boosted model,
   $$\hat{f}(x) = \sum_{b=1}^{B} \lambda \hat{f}^b(x). \tag{8.12}$$

Source: [James et al., 2013]. The figure denotes shrinkage by λ; our notation uses ν.

Three ways to use trees

Single tree

- **Build:** partition recursively
- **Goal:** interpret nonlinear structure
- **Control:** tree size and pruning

Bagging / forest

- **Build:** fit many trees and average
- **Goal:** reduce variance and correlation
- **Control:** s, m, and leaf size k

Boosting

- **Build:** add trees sequentially
- **Goal:** improve the current fit
- **Control:** depth, M, ν, and early stopping

Application: does division imply predictability?

Kim and Zilinsky: *Division Does Not Imply Predictability* [Kim and Zilinsky, 2024]

Research question

If Americans are increasingly sorted into demographic political camps, should demographics predict vote choice and party ID more accurately over time?

Data

- American National Election Studies, 1952–2020
- Outcomes: two-party presidential vote and binary party identification
- Basic predictors: age, gender, race, education, and income

From association to prediction

- Particular demographics can have sizable and statistically significant marginal effects.
- Social sorting makes a stronger joint prediction: demographic labels should reliably classify political behavior.

Random forests as a test of social sorting

Demographic characteristics $\longrightarrow$ random forest or logit $\longrightarrow$ held-out vote / party ID

Evaluation design

- Estimate each ANES wave separately.
- Use 80% for training and 20% for testing.
- Tune two forest hyperparameters by ten-fold cross-validation inside the training sample.

Information sets

- Basic: five demographic characteristics (age, gender, race, education, and income)
- Extended: additional demographic labels (employment, religious, homeownership, etc)
- Richer vote models: add party ID, issues, and other survey items

Key idea

Comparing a flexible forest with logit tests whether nonlinearities and interactions contain

Division does not imply predictability

Predictors / outcome	RF	Logit
Basic / vote choice	63.9%	64.5%
Basic / party ID	63.4%	63.9%
Extended / vote choice	67.4%	66.8%
Extended / party ID	67.2%	67.0%

Mean test accuracy across ANES waves; [Kim and Zilinsky, 2024].

What the comparison reveals

- Demographic accuracy is modest and stable from 1952 to 2020.
- Random forests match logit: extra flexibility does not change the conclusion.
- Adding party ID raises vote-choice accuracy above 90% in most elections since 1996.

Lesson

Group differences can be real without making individuals highly predictable. Demographic division is not the same as demographic determinism.

Application: predicting stock returns

Gu, Kelly, and Xiu: *Empirical Asset Pricing via Machine Learning* [Gu et al., 2020]

Research question

Can high-dimensional machine learning improve our measurement of expected stock returns?

Data

- Monthly CRSP returns for nearly 30,000 U.S. stocks, 1957–2016
- Ninety-four firm characteristics, eight macro predictors, and industry indicators
- Their interactions yield more than 900 potential predictors

Why is this difficult?

- Expected returns are a weak signal amid unpredictable news.
- Nonlinearities and interactions may matter.
- With hundreds of predictors, unrestricted OLS can overfit badly.

A disciplined out-of-sample comparison

Firm characteristics $\times$ macroeconomic conditions $\longrightarrow$ competing models $\longrightarrow \widehat{r}_{i,t+1}$

Time-based sample split

- **Training:** 1957–1974
- **Validation:** 1975–1986
- **Testing:** 1987–2016
- Models are re-estimated annually as the historical training sample expands.

Models compared

- OLS and dimension reduction
- Elastic net and generalized linear models
- Random forests and boosted trees
- Neural networks of different depths

Key idea

Validation chooses model complexity; the test period evaluates whether the learned relationship survives in the future.

What do tree methods learn about returns?

Model	Monthly R^2_{oos}
OLS-3	0.16%
Elastic net	0.11%
Random forest	0.33%
Boosted trees	0.34%
Neural network (3 layers)	0.40%

All stocks, monthly test predictions;
[Gu et al., 2020], Table 1.

Substantive results

- Tree methods substantially outperform sparse linear benchmarks, although neural networks perform best.
- Their advantage comes largely from nonlinear interactions among predictors.
- Momentum, liquidity, and volatility emerge as dominant predictive signals across methods.

Interpretation

A small out-of-sample R^2 can be economically meaningful when returns are extremely noisy. These are predictive relationships, not causal effects of firm characteristics.

References

Gu, S., Kelly, B., and Xiu, D. (2020).
Empirical asset pricing via machine learning.
The review of financial studies, 33(5):2223–2273.

James, G., Witten, D., Hastie, T., and Tibshirani, R. (2013).
An introduction to statistical learning: with applications in R, volume 103.
Springer.

Kim, S.-y. S. and Zilinsky, J. (2024).
Division does not imply predictability: Demographics continue to reveal little about voting and partisanship.
Political behavior, 46(1):67–87.

Wager, S. and Athey, S. (2018).
Estimation and inference of heterogeneous treatment effects using random forests.
Journal of the American Statistical Association, 113(523):1228–1242.